

TopShelf Operating System

Foundation Contract Package

Formal Decision and Zero-Ambiguity Contract Baseline

Version 2.0.0

Decision date: 2026-07-23

Owner: Patrick Craven

Company: Top Shelf Service LLC

CONTROLLED BASELINE - SUPERSEDES TOS FORMAL DECISION PACKAGE v1.0

Document Control

Field	Value
Official name	TopShelf Operating System
Short name	TOS
Version	2.0.0
Status	Approved foundation baseline
Owner	Patrick Craven
Company	Top Shelf Service LLC
Supersedes	TOS Formal Decision Package v1.0
Contract count	105
Primary repository	topshelf-operating-system
CLI	tos

Governing Decision

The official product name is TopShelf Operating System. TOS is the official short name. The earlier expansion "TopShelf Op Sys" is superseded. TOS is a project operating system for a solo owner working with AI agents, not merely a project template or kernel.

Every foundation concern is named in the contract catalog. Each project must evaluate every contract as required, conditional, or not applicable with a recorded reason and authority. Omission is a validation failure.

How to Use This Package

1. Adopt the official terminology and identifier rules.
2. Use the common contract envelope for every instantiated contract.
3. Evaluate the entire catalog during project boot and whenever project facts change.
4. Generate only applicable contract instances, but retain an applicability register for all catalog entries.
5. Bind tasks, reviews, evidence, promotions, recovery, and upgrades to the applicable contracts.
6. Propagate generalizable project lessons back into TOS controls and negative fixtures.

1. Official Identity and Controlled Vocabulary

Item	Controlled value
Product name	TopShelf Operating System
Short name	TOS
Repository	topshelf-operating-system
CLI	tos
State directory	.tos/
Contract ID	TOS-CTR-###
Decision ID	TOS-DEC-###
Requirement ID	TOS-REQ-###
Evidence ID	TOS-EVD-###
Task ID	<PROJECT>-<WORKSTREAM>-###
Package prefix	@topshelf/tos-<capability>

Controlled terms

Term	Definition
TopShelf Operating System	The official product name. The only permitted expansion of TOS.
TOS	The official short name for TopShelf Operating System.
Kernel	Stable schemas, state model, policy evaluation, contract system, and validation primitives.

Term	Definition
Runtime	Executable services and CLI that inspect, schedule, constrain, verify, promote, recover, and upgrade projects.
Module	Versioned package of contracts, controls, workflows, and checks activated by project facts.
Adapter	Provider-specific implementation of an agnostic TOS capability.
Project instance	A repository or initiative governed by a specific TOS version and project manifest.
Contract	A versioned, owned, testable statement of obligations, permissions, prohibitions, failure behavior, and evidence.
Contract catalog	The complete list of named foundation concerns that every project must evaluate.
Project fact	Evidence-backed record of current and intended project state.
Truth engine	TOS subsystem that inspects evidence, compares declarations, classifies facts, detects conflict, and asks targeted questions.
Task graph	Machine-readable dependency graph of work, decisions, blockers, and gates.
Vertical slice	A bounded end-to-end unit of value or control that can be independently implemented and proven.
Work order	Compiled execution contract for a human or AI function.
Task lease	Time-bounded claim on a task and resource scope.
Evidence	Version-bound proof supporting a specific claim and evidence class.
Promotion	Controlled transition from one lifecycle state to another.
Negative control	Deliberate invalid condition proving that a checker or gate can fail correctly.
Owner	The human with final business authority: Patrick Craven unless formally delegated.
Function	An accountable operating responsibility; it may be performed by Patrick, an AI agent, or another authorized person.
Canonical state	Authoritative human-readable records under <code>`.tos/`</code> .
Derived state	Rebuildable indexes, caches, summaries, or views that are not authoritative.
Fail closed	Deny or block when authority, scope, state, evidence, or applicability cannot be determined safely.
Not applicable	An explicit evaluated disposition with recorded reason and authority; never an omitted contract.

2. System Definition and Architecture

TOS owns the transition from one valid project state to the next. Its governing loop is: observe, determine truth, decide what is allowed, select work, constrain execution, verify results, update state, permit or deny promotion, recover from failure, and learn.

Layer	Responsibility
Human authority	Final business, financial, risk, external-authorization, promotion, and override decisions.
Kernel	Schemas, state model, vocabulary, policies, contract registry, compatibility rules, validation primitives.
Runtime	CLI, inspection, intake, truth resolution, scheduler, work orders, leases, evidence, promotion, recovery, upgrades.
Modules	Versioned controls and workflows activated by project facts.
Adapters	Provider-specific implementations of agnostic capabilities.
Project instance	Canonical <code>`.tos/`</code> state and project-specific contract instances.
Derived index	Rebuildable SQLite cache and generated views; never authoritative.

Canonical state files

- `.tos/project.yaml`
- `.tos/facts.yaml`
- `.tos/modules.yaml`
- `.tos/task-graph.yaml`
- `.tos/decisions.yaml`
- `.tos/blockers.yaml`

- .tos/claims.yaml
- .tos/evidence-index.yaml
- .tos/promotions.yaml
- .tos/activity.jsonl

Implementation baseline

- TypeScript on Node.js 20 or later.
- pnpm workspace and Semantic Versioning.
- YAML canonical records validated by JSON Schema.
- SQLite as a rebuildable local index.
- Append-only JSON Lines activity log.
- Provider adapters outside the kernel.
- Least-privilege tool and resource scopes.

3. Constitutional Rules

1. TOS inspects before asking and never requires the owner to know a technical fact that can be verified automatically.
2. Project facts separate observed current state from intended target state.
3. Absence of evidence never proves a material fact false.
4. Material unknowns and conflicts fail closed and activate conservative controls.
5. Every requirement, task, milestone, release, and promotion has observable acceptance criteria.
6. Every material checker and gate has a negative control.
7. Implementers may not approve their own work.
8. Evidence classes remain distinct and are bound to exact versions.
9. Lifecycle labels change only through the promotion controller.
10. External authorization, legal approval, risk acceptance, and financial authority cannot be inferred from code.
11. Deployable and migratable changes require rollback; irreplaceable state requires tested restore.
12. Every lesson is evaluated for reusable TOS propagation.

4. Contract Model

TOS defines 105 named foundation contracts. Every template uses the same envelope so tools and agents can reason about ownership, obligations, enforcement, evidence, compatibility, and retirement consistently.

Common contract envelope

Section	Mandatory contents
Identity	ID, type, title, version, status, applicability, owner, approver, applies-to, dates, supersession, source of truth.
Definition	Purpose, scope, exclusions, terms, trigger.
Obligations	Inputs, outputs, preconditions, invariants, postconditions, permissions, prohibitions, failure behavior.
Assurance	Acceptance criteria, validation, negative controls, evidence.
Change and exit	Change control, compatibility, deprecation, retirement.
Contract body	Contract-specific fields listed in the catalog template.

Applicability rule

Disposition	Meaning
required	The contract must be instantiated and active.
conditional	The contract is required when its trigger is true or unresolved with material risk.
not_applicable	The project records a reason, approving authority, and review trigger.

5. Complete Foundation Contract Catalog

The complete machine-readable register is included as `registers/contract-register.csv`; the individual agnostic templates are under `contracts/`.

Foundation and Governance

ID	Contract	Default	Trigger	Owner function
TOS-CTR-001	System Charter	core	Every TOS installation and project	Owner
TOS-CTR-002	Constitutional Principles	core	Every TOS installation	Owner
TOS-CTR-003	Naming and Terminology	core	Every TOS installation and project	Knowledge Manager
TOS-CTR-004	Authority and Approval	core	Every TOS installation and project	Owner
TOS-CTR-005	Source of Truth	core	Every project	Project Manager
TOS-CTR-006	Project Manifest	core	Every project	Project Manager
TOS-CTR-007	Project Fact	core	Every fact used for module selection or gating	Research Discovery
TOS-CTR-008	Assumption	core	Any unresolved premise affecting scope, design, schedule, risk, or release	Project Manager
TOS-CTR-009	Decision	core	Any material choice or binding interpretation	Owner
TOS-CTR-010	Risk	core	Any uncertain event with material impact	Project Manager
TOS-CTR-011	Issue	core	Any realized problem requiring resolution	Project Manager
TOS-CTR-012	Change Control	core	Any change to approved scope, architecture, schedule, cost, risk, or contract	Project Manager
TOS-CTR-013	Dependency	core	Any work or decision dependent on another internal or external item	Project Manager
TOS-CTR-014	Requirement	core	Every committed product, system, control, or operational obligation	Product Manager
TOS-CTR-015	Acceptance Criteria	core	Every requirement, feature, task, milestone, and release	Product Manager
TOS-CTR-016	Definition of Done	core	Every work class and lifecycle promotion	Project Manager
TOS-CTR-017	Milestone	core	Every material lifecycle or delivery checkpoint	Project Manager
TOS-CTR-018	Promotion	core	Every lifecycle-state transition	Release Ops
TOS-CTR-019	Retirement	conditional	Any project, module, component, service, contract, or data set being ended	Operations Manager

Planning and Delivery

ID	Contract	Default	Trigger	Owner function
TOS-CTR-020	Roadmap	core	Every project	Product Manager
TOS-CTR-021	Work Breakdown Structure	core	Every nontrivial project or milestone	Project Manager
TOS-CTR-022	Task or Vertical Slice	core	Every executable unit of work	Project Manager
TOS-CTR-023	Agent Work Order	conditional	Any task executed or reviewed by an AI agent	Workflow Coordinator
TOS-CTR-024	Task Lease	conditional	Any concurrently executable or agent-owned task	Workflow Coordinator
TOS-CTR-025	Handoff	core	Any transfer between	Workflow Coordinator

ID	Contract	Default	Trigger	Owner function
			functions, agents, phases, or owners	
TOS-CTR-026	Review	core	Any material deliverable requiring independent evaluation	QA Test Engineer
TOS-CTR-027	Blocker	core	Any condition preventing valid progress	Project Manager
TOS-CTR-028	Schedule	conditional	Any project with time-bound commitments or dependencies	Project Manager
TOS-CTR-029	Release	conditional	Any versioned or delivered product increment	Release Ops
TOS-CTR-030	Rollback	conditional	Any deployable, migratable, or externally delivered change	Release Ops
TOS-CTR-031	Closeout	core	Every completed task, milestone, release, or project	Project Manager
TOS-CTR-032	Lesson and Improvement	core	Every material success, failure, incident, or retrospective	Knowledge Manager

Architecture and Software

ID	Contract	Default	Trigger	Owner function
TOS-CTR-033	System Architecture	conditional	Every software, technical, or multi-component project	Solution Architect
TOS-CTR-034	Domain	conditional	Any project with domain rules or business state	Solution Architect
TOS-CTR-035	Component	conditional	Every independently owned runtime, package, app, service, or major subsystem	Solution Architect
TOS-CTR-036	Package Boundary	conditional	Any multi-package or modular codebase	Solution Architect
TOS-CTR-037	Repository	conditional	Every source-controlled project	Infrastructure DevOps
TOS-CTR-038	Branch Commit and Pull Request	conditional	Any source-controlled change	Infrastructure DevOps
TOS-CTR-039	Software Dependency	conditional	Any third-party or internal software dependency	Solution Architect
TOS-CTR-040	Data and Schema	conditional	Any persisted or exchanged data	Database Engineer
TOS-CTR-041	State Machine	conditional	Any lifecycle, workflow, or status-bearing entity	Solution Architect
TOS-CTR-042	Transaction	conditional	Any multi-step operation requiring atomicity or idempotency	Database Engineer
TOS-CTR-043	API	conditional	Any callable interface between systems or components	Backend API Engineer
TOS-CTR-044	Event and Message	conditional	Any asynchronous event, queue, webhook, or message	Backend API Engineer
TOS-CTR-045	Service	conditional	Any independently operated service	Solution Architect
TOS-CTR-046	Integration	conditional	Any connection to an external or separately owned system	Solution Architect
TOS-CTR-047	Provider Adapter	conditional	Any provider-specific implementation behind an agnostic capability	Infrastructure DevOps
TOS-CTR-048	Environment	conditional	Any local, test, staging, pilot, or production environment	Infrastructure DevOps
TOS-CTR-049	Deployment	conditional	Any deployed artifact or service	Infrastructure DevOps
TOS-CTR-050	Migration	conditional	Any data, architecture, provider, or component transition	Solution Architect
TOS-CTR-051	Deprecation	conditional	Any interface, feature, component, module, or contract being phased out	Solution Architect
TOS-CTR-052	UI Route and Surface	conditional	Any user-visible route, screen, or interaction surface	Frontend Engineer
TOS-CTR-053	Design System and Brand	conditional	Any branded or user-facing project	Marketing Positioning
TOS-CTR-054	Content Pack and Engine Fuel	conditional	Any system separating reusable logic from customer or domain content	Content Engineer
TOS-CTR-055	Configuration	conditional	Any configurable system behavior	Infrastructure DevOps

ID	Contract	Default	Trigger	Owner function
TOS-CTR-056	Feature Flag	conditional	Any controlled rollout, experiment, or disabled path	Product Manager

Security, Privacy, and Identity

ID	Contract	Default	Trigger	Owner function
TOS-CTR-057	Security Policy	conditional	Any software, data, identity, provider, or external access	Security Privacy
TOS-CTR-058	Threat Model	conditional	Any system with assets, trust boundaries, users, data, or external exposure	Security Privacy
TOS-CTR-059	Data Classification	conditional	Any stored, processed, or transmitted data	Security Privacy
TOS-CTR-060	Privacy	conditional	Any personal, sensitive, behavioral, or customer data	Security Privacy
TOS-CTR-061	Identity Lifecycle	conditional	Any human or machine identity	Security Privacy
TOS-CTR-062	Credential and Secret	conditional	Any password, key, token, certificate, or secret	Security Privacy
TOS-CTR-063	Authorization Capability	conditional	Any protected action, resource, or administrative function	Security Privacy
TOS-CTR-064	Tenant and Scope	conditional	Any multi-organization, multi-location, or scoped-access system	Security Privacy
TOS-CTR-065	Audit Log	conditional	Any security, compliance, administrative, financial, or critical domain operation	Security Privacy
TOS-CTR-066	Data Retention and Deletion	conditional	Any persisted data	Security Privacy
TOS-CTR-067	Security Vulnerability	conditional	Any discovered weakness or security finding	Security Privacy
TOS-CTR-068	Compliance and Approval	conditional	Any legal, regulatory, contractual, privacy, or security approval requirement	Legal Contracts Coordinator

Operations and Evidence

ID	Contract	Default	Trigger	Owner function
TOS-CTR-069	Evidence	core	Every completed claim, task, gate, milestone, release, and promotion	Release Ops
TOS-CTR-070	Validation	core	Every contract and deliverable	QA Test Engineer
TOS-CTR-071	Test Strategy	conditional	Any executable or behavior-bearing system	QA Test Engineer
TOS-CTR-072	Negative Control	core	Every important checker, gate, policy, and safety control	QA Test Engineer
TOS-CTR-073	Observability	conditional	Any operated software, service, workflow, or agent runtime	Observability Support
TOS-CTR-074	SLI and SLO	conditional	Any production, pilot, or reliability-committed service	Observability Support
TOS-CTR-075	Alert	conditional	Any monitored condition requiring action	Observability Support
TOS-CTR-076	Incident	conditional	Any material service, security, data, agent, or delivery failure	Operations Manager
TOS-CTR-077	Postmortem	conditional	Any qualifying incident or repeated failure	Operations Manager
TOS-CTR-078	Backup	conditional	Any irreplaceable data, configuration, evidence, or state	Infrastructure DevOps
TOS-CTR-079	Restore	conditional	Any backup-dependent system	Infrastructure DevOps
TOS-CTR-080	Runbook and Support	conditional	Any operated, handed-off, or externally used system	Knowledge Manager
TOS-CTR-081	Cost and Vendor	conditional	Any material provider, vendor, subscription, labor, or operational cost	Cost Pricing
TOS-CTR-082	Business Continuity	conditional	Any service or process whose interruption is material	Operations Manager
TOS-CTR-083	External Authorization	conditional	Any client, participant, provider, legal, or external approval gate	Legal Contracts Coordinator

ID	Contract	Default	Trigger	Owner function
TOS-CTR-084	Public Claim	conditional	Any public, client-facing, sales, legal, privacy, security, or readiness statement	Marketing Positioning

AI and Agent Control

ID	Contract	Default	Trigger	Owner function
TOS-CTR-085	Agent Role	conditional	Any AI agent or automated function	Workflow Coordinator
TOS-CTR-086	Agent Authority	conditional	Any AI agent allowed to take actions	Owner
TOS-CTR-087	Agent Tool Permission	conditional	Any AI agent with tool or connector access	Security Privacy
TOS-CTR-088	Agent Work Context	conditional	Any bounded AI task	Workflow Coordinator
TOS-CTR-089	Prompt and Instruction	conditional	Any reusable agent prompt, system instruction, or skill	Knowledge Manager
TOS-CTR-090	Agent Memory	conditional	Any retained agent or project context	Security Privacy
TOS-CTR-091	Agent Evaluation	conditional	Any agent role, workflow, prompt, or tool policy	QA Test Engineer
TOS-CTR-092	Agent Handoff	conditional	Any AI-to-AI or AI-to-human transfer	Workflow Coordinator
TOS-CTR-093	Agent Review Separation	conditional	Any AI-generated material decision or implementation	QA Test Engineer
TOS-CTR-094	Agent Failure and Retry	conditional	Any automated or agent-executed process	Workflow Coordinator
TOS-CTR-095	Agent Audit	conditional	Any agent action with material impact	Security Privacy
TOS-CTR-096	Human Override	core	Every TOS-controlled automated action	Owner

Commercial, Product, and Client

ID	Contract	Default	Trigger	Owner function
TOS-CTR-097	Product Outcome	conditional	Any product, service, client engagement, or internal initiative	Product Manager
TOS-CTR-098	User and Actor	conditional	Any system with human, organizational, machine, or external actors	Product Manager
TOS-CTR-099	Customer and Organization Lifecycle	conditional	Any client, tenant, customer, location, or organizational account	Client Success Implementation
TOS-CTR-100	Entitlement and Billing	conditional	Any paid, limited, metered, licensed, or tiered capability	Cost Pricing
TOS-CTR-101	Offer and Service Tier	conditional	Any packaged product, service, subscription, or engagement level	Product Manager
TOS-CTR-102	Client Contract and Statement of Work	conditional	Any client-facing paid or obligated engagement	Legal Contracts Coordinator
TOS-CTR-103	Pilot	conditional	Any real-user trial, limited deployment, or evaluation	Project Manager
TOS-CTR-104	Consent and Participation	conditional	Any participant notice, consent, agreement, or voluntary study activity	Legal Contracts Coordinator
TOS-CTR-105	Pricing and Budget	conditional	Any project with cost, price, revenue, budget, or financial exposure	Cost Pricing

6. Controlled States and Enumerations

Contract Status

`draft`, `proposed`, `approved`, `active`, `superseded`, `retired`

Applicability

`required`, `conditional`, `not\_applicable`

Truth Status

`verified\_yes`, `verified\_no`, `declared\_yes`, `declared\_no`, `inferred\_yes`, `inferred\_no`, `unknown`, `conflicting`, `stale`, `not\_applicable`

Task Status

`backlog`, `ready`, `leased`, `active`, `review`, `verification`, `blocked`, `done`, `cancelled`

Lifecycle State

`idea`, `discovery`, `prototype`, `internal\_demo`, `release\_candidate`, `authorized\_pilot`, `production`, `general\_availability`, `retired`

Evidence Class

`automated`, `local\_manual`, `deployed\_delivered`, `external\_approval`

Priority

`P0`, `P1`, `P2`, `P3`

Severity

`SEV0`, `SEV1`, `SEV2`, `SEV3`, `SEV4`

Confidence

`high`, `medium`, `low`, `none`

Decision Status

`proposed`, `approved`, `rejected`, `superseded`, `withdrawn`

7. Truth and Interactive Intake

TOS resolves project truth through deterministic inspection, operational verification, authoritative declarations, and explainable AI inference. AI inference may recommend but may not silently settle material facts.

Evidence level	Use	Authority
Deterministic repository evidence	Dependencies, routes, schemas, configuration, workflows, source boundaries.	May verify current technical state.
Operational evidence	Live deployments, providers, migrations, alerts, restore, runtime behavior.	May verify named environment and version.
Human or external declaration	Intent, ownership, authorization, legal approval, pricing, risk acceptance.	Required for nontechnical authority facts.
AI inference	Interpretation, contradiction detection, targeted follow-up.	Advisory; never sole authority for material facts.

Resolution algorithm

1. Classify the question as current state, target intent, external authorization, or judgment.
2. Run deterministic detectors.
3. Gather operational evidence when available.
4. Compare evidence with existing declarations.
5. Classify as verified, declared, inferred, unknown, conflicting, stale, or not applicable.
6. Ask one targeted question only when unresolved information requires human judgment.

7. Apply conservative controls to material uncertainty.
8. Record evidence, confidence, source, verification time, and invalidation conditions.
9. Reconcile modules, contracts, tasks, blockers, and promotion eligibility.

8. Solo Owner and AI Agent Operating Model

Function	Accountability	Prohibited authority
Owner	Direction, risk, finances, external authorization, final promotion, override.	May not claim technical evidence that was not produced.
Workflow Coordinator	Scheduling, leases, handoffs, shared state reconciliation.	May not implement or grant final approval.
Project Manager	Scope, requirements, risks, changes, acceptance, closeout.	May not unilaterally override architecture or external authorization.
Implementer	Bounded implementation and targeted tests.	May not approve own work or expand scope silently.
Independent Reviewer	Contract and architecture review.	May not substitute review for execution evidence.
Evidence Verifier	Reproduction, evidence classification, exact-version checks.	May not infer legal, customer, or owner approval.
Release Ops	Promotion readiness, rollback, recovery, release evidence.	May not promote without all required approvals.

Mandatory agent controls

- Agent role
- Agent authority
- Agent tool permission
- Agent work context
- Agent work order
- Task lease
- Agent handoff
- Independent review separation
- Agent evaluation
- Failure and retry
- Agent audit
- Human override

9. Evidence, Promotion, and Recovery

Evidence class	What it can prove	What it cannot prove
automated	Named checks passed for a named version and environment.	Deployed behavior, manual use, external authorization.
local_manual	Recorded behavior on the stated local environment.	Live deployment or provider behavior.
deployed_delivered	Behavior of the named deployment or delivered artifact.	Legal, customer, or owner authorization.
external_approval	The recorded approval and its stated scope.	Technical correctness beyond the approval evidence.

Lifecycle states

idea -> discovery -> prototype -> internal_demo -> release_candidate -> authorized_pilot -> production -> general_availability -> retired

Recovery requirements

- Last known-good version is recorded.
- Rollback trigger and authority are explicit.
- Data implications and irreversible steps are identified.
- Backup exists for irreplaceable state.
- Restore is tested at a declared frequency.
- Incidents have severity, command, containment, communication, recovery, and closure.
- Qualifying incidents produce postmortems and tracked corrective controls.

10. Module, Adapter, and Upgrade Model

Object	Required contract contents
Module	Name, version, provides, requires, recommends, conflicts, activation facts, contracts, checks, migrations, removal rules.
Adapter	Agnostic capability, provider mapping, configuration, credentials, failure translation, portability tests, exit strategy.
Upgrade	Source and target TOS versions, ownership classification, migrations, conflicts, skipped changes, validation, rollback.

Downstream ownership rule

- Unchanged TOS-owned files may be updated automatically.
- Modified TOS-owned files require a three-way merge and decision.
- Project-owned files are never overwritten automatically.
- Skipped or deferred controls require a recorded upgrade decision.
- Module removal is denied while consumers or required contracts remain.

11. Teach Strategy

Teach remains the product and authoritative behavioral reference. TOS may generate a clean Teach foundation reference, but Teach is not replaced until bounded slices prove parity, exact-version evidence, deployed behavior, rollback, and safe cutover.

1. Define binding domain and component contracts.
2. Create parity and negative tests against current behavior.
3. Implement one bounded replacement boundary.
4. Verify locally and against the exact deployment version.

5. Release behind isolation or a disabled flag.
6. Prove rollback.
7. Remove the old path only after parity and owner approval.
8. Capture generalizable lessons into TOS controls and upgrades.

12. Roadmap

Release	Outcome	Required commands
0.1	Kernel consolidation: CLI, canonical state, schemas, contract registry, module manifests, status and validation.	tos status; tos validate; tos module list
0.2	Truth and boot: inspectors, intake, fact resolution, module selection, generation.	tos inspect; tos intake; tos truth
0.3	Scheduler and agent control: task graph, next work, work orders, leases, handoffs.	tos next; tos task compile; tos task claim; tos task handoff
0.4	Evidence and assurance: exact-version evidence, semantic validation, negative controls.	tos verify; tos evidence show
0.5	Promotion and recovery: lifecycle gates, rollback, incidents, restore.	tos promote; tos rollback; tos recover
1.0	Modules, adapters, downstream upgrades, lessons, Teach reference, self-tests.	tos module upgrade; tos upgrade; tos learn

13. Approved Decision Register

ID	Decision	Status
TOS-DEC-001	TopShelf Operating System is the official product name; TOS is the short name.	approved
TOS-DEC-002	TOS is a project operating system, not merely a template or kernel.	approved
TOS-DEC-003	The core repository is named topshelf-operating-system.	approved
TOS-DEC-004	The primary command is tos.	approved
TOS-DEC-005	TypeScript, Node.js 20+, and pnpm are the default implementation stack.	approved
TOS-DEC-006	Canonical project state is YAML under .tos; SQLite is a rebuildable derived index.	approved
TOS-DEC-007	Every catalog contract is evaluated as required, conditional, or not applicable with reason.	approved
TOS-DEC-008	Project answers are evidence-backed records, not plain booleans.	approved
TOS-DEC-009	TOS inspects first, asks only unresolved questions, and defaults conservatively on material uncertainty.	approved
TOS-DEC-010	Patrick retains final risk, external authorization, promotion, financial, and merge authority unless delegated.	approved
TOS-DEC-011	Implementers may not approve their own work; review and evidence verification are separate functions.	approved
TOS-DEC-012	Automated, local manual, deployed/delivered, and external approval evidence remain distinct.	approved
TOS-DEC-013	Lifecycle labels may change only through the promotion controller and required evidence.	approved
TOS-DEC-014	Teach remains the product reference; migration uses bounded strangler slices, not a blind rewrite.	approved
TOS-DEC-015	Project lessons must be evaluated for propagation into reusable TOS controls.	approved

14. Foundational Definition of Done

1. The official name and all identifier rules are consistent.
2. Every foundational term is defined.
3. Every catalog contract has an agnostic template and common envelope.
4. Every project must disposition every catalog entry.
5. Core schemas validate project, fact, task, evidence, module, decision, promotion, and work-order records.
6. Controlled states and enums are specified.
7. Human authority and AI-agent separation are explicit and enforceable.
8. Truth, scheduling, evidence, promotion, recovery, module, adapter, upgrade, and learning behavior are named and contracted.
9. Teach strategy and roadmap are formalized.
10. Published artifacts are checksum-controlled and visually verified.

15. Package Inventory

Path	Purpose
README.md	Package orientation and implementation defaults.
docs/	Formal source documents.
contracts/	Agnostic templates for every named foundation contract.
schemas/	JSON Schemas for core records and common contract envelope.
registers/contract-register.csv	Complete contract catalog and applicability triggers.
registers/term-register.csv	Controlled vocabulary.
registers/decision-register.csv	Approved foundation decisions.
registers/requirement-register.csv	Initial TOS requirements.
examples/	Filled examples of core records.
visuals/	Mermaid architecture and lifecycle source.
TOS_PACKAGE_MANIFEST.json	Checksums and package metadata.